

DATA 260 Homework 5 Report

GITHUB REPOSITORY: <https://github.com/rutujasaste06/data260-5825>

Student ID- 020065825

SID4- 5825

PORT_BASE - 8425

PREFIX - s5825

SEED - 5825

VERIFY_SEED – 265825

DOMAIN_ID - 1

DOMAIN - Clinical trial listings

HARDWARE - MacBook Pro, Apple M5 chip

LOCAL MODEL USED: qwen3:8b (Ollama)

TAGGED COMMIT HASH: f46cb2c928feef9424af34f21dfc46374c0681a9

Note: For part 3,4 and 5 I used 13th Gen Intel Core i7-13700HX (2.10 GHz), 16GB RAM, NVIDIA GeForce RTX 4070 Laptop GPU (8GB)

Outputs:

PART 1:

I. Database

Created new table “sponsors” that will be linked to the “trials” table. Hence modified the “trials” table to add few more columns. Here “id” from “sponsors” table works as foreign key from “trials” table.

```
ALTER TABLE trials ADD COLUMN updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP; --
(venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % mysql -u root -p s5825_rel -e "
SHOW TABLES;
DESCRIBE sponsors;
DESCRIBE trials;
"
Enter password:
+-----+
| Tables_in_s5825_rel |
+-----+
| sessions            |
| sponsors             |
| trial_sites         |
| trials              |
| users               |
+-----+

+-----+
| Field          | Type          | Null | Key | Default | Extra          |
+-----+
| id             | int           | NO   | PRI | NULL    | auto_increment |
| sponsor_name   | varchar(255)  | NO   |     | NULL    |                |
| contact_person | varchar(255)  | NO   |     | NULL    |                |
| email          | varchar(255)  | NO   | UNI | NULL    |                |
| created_at     | datetime      | YES  |     | now()    | DEFAULT_GENERATED |
| updated_at     | datetime      | YES  |     | now()    | DEFAULT_GENERATED |
+-----+

+-----+
| Field          | Type          | Null | Key | Default | Extra          |
+-----+
| id             | int           | NO   | PRI | NULL    | auto_increment |
| trial_title    | varchar(255)  | NO   |     | NULL    |                |
| nct_number     | varchar(50)   | NO   | MUL | NULL    |                |
| available_slots | int           | NO   |     | 10      |                |
| sponsor_id     | int           | YES  | MUL | NULL    |                |
| created_at     | datetime      | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
| updated_at     | datetime      | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED on update CURRENT_TIMESTAMP |
+-----+

(venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 %
```

Updated the swagger page that also includes the sponsor API's

POST	/auth/Login	Login
GET	/auth/me	Me
POST	/auth/Logout	Logout
POST	/sponsors	Add Sponsor
GET	/sponsors	List Sponsors
GET	/sponsors/{sponsor_id}	Get Sponsor
PUT	/sponsors/{sponsor_id}	Edit Sponsor
DELETE	/sponsors/{sponsor_id}	Remove Sponsor
GET	/sponsors/{sponsor_id}/trials	Trials For Sponsor
GET	/trials-naive	Trials Naive
GET	/trials-fixed	Trials Fixed
GET	/trials	List Trials
POST	/trials	Add Trial
GET	/trials/{trial_id}	Get Trial
PUT	/trials/{trial_id}	Edit Trial
DELETE	/trials/{trial_id}	Remove Trial

II. API CRUD operations :

a. POST: Login using existing user email id and password

HTTP HW4-s5825 > 2 - LOGIN Save Share

POST `http://127.0.0.1:8425/auth/login` Send

Docs Params Authorization Headers 9 **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** Schema Beautify

```

1  {"email": "user1@gmail.com", "password": "secret123"}

```

Body Cookies 1 Headers 5 Test Results 200 OK • 210 ms • 312 B Save Response

{} JSON Preview Visualize

```

1  {
2    "message": "logged in",
3    "user_id": 1,
4    "name": "User1"
5  }

```

b. POST- Creating first sponsor

HTTP HW4-s5825 > **POST - Create your first sponsor** Save Share

POST http://127.0.0.1:8425/sponsors Send

Docs Params Authorization Headers 9 **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** Schema Beautify

```
1 {"sponsor_name": "Pfizer Research", "contact_person": "Dr. Sarah Chen", "email": "sarah.chen@pfizer.com"}
```

Body Cookies 1 Headers 4 Test Results 200 OK • 17 ms • 305 B • Save Response

{ } JSON Preview Visualize

```
1 {
2   "id": 1,
3   "sponsor_name": "Pfizer Research",
4   "contact_person": "Dr. Sarah Chen",
5   "email": "sarah.chen@pfizer.com",
6   "created_at": "2026-10-02T18:26:38",
7   "updated_at": "2026-10-02T18:26:38"
8 }
```

c. Create a trial linked to this sponsor

HTTP HW4-s5825 > **POST - CREATE TRIAL linked to the sponsor** Save Share

POST http://127.0.0.1:8425/trials Send

Docs Params Authorization Headers 9 **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** Schema Beautify

```
1 {"trial_title": "Pfizer Diabetes Phase III", "nct_number": "NCT99999001", "available_slots": 25, "sponsor_id": 1}
```

Body Cookies 1 Headers 4 Test Results 200 OK • 15 ms • 314 B • Save Response

{ } JSON Preview Visualize

```
1 {
2   "id": 5001,
3   "trial_title": "Pfizer Diabetes Phase III",
4   "nct_number": "NCT99999001",
5   "available_slots": 25,
6   "sponsor_id": 1,
7   "created_at": "2026-10-02T18:29:02",
8   "updated_at": "2026-10-02T18:29:02"
9 }
```

d. GET – Get all trials for sponsor 1

HW4-s5825 > GET - Get all trials for sponsor 1

GET http://127.0.0.1:8425/sponsors/1/trials

Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
☐ id	1			
Key	Value	Description		

Body Cookies 1 Headers 4 Test Results

200 OK • 5 ms • 316 B • Save Response

{ } JSON Preview Visualize

```
1 [
2   {
3     "id": 5001,
4     "trial_title": "Pfizer Diabetes Phase III",
5     "nct_number": "NCT99999001",
6     "available_slots": 25,
7     "sponsor_id": 1,
8     "created_at": "2026-10-02T18:29:02",
9     "updated_at": "2026-10-02T18:29:02"
10  }
11 ]
```

- e. DELETE - Deleting the sponsor 1 (which is associated with the trial id : 5001). This is handled by sending 409 error showing the error message as Cannot delete sponsor: as trials are still linked to it.

HW4-s5825 > DELETE - Deleting the sponsor ID directly

DELETE http://127.0.0.1:8425/sponsors/1

Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies 1 Headers 4 Test Results

409 Conflict • 11 ms • 198 B • Save Response

Handle linked trials before deletion Diagnose Decline

{ } JSON Preview Debug with AI

```
1 {
2   "detail": "Cannot delete sponsor: trials are still linked to it"
3 }
```

Code that handles this:

```
@app.delete("/sponsors/{sponsor_id}", response_model=schema.SponsorOut)
def remove_sponsor(sponsor_id: int, db: Session = Depends(get_db),
                  _s=Depends(require_session)):
    sponsor, reason = crud.delete_sponsor(db, sponsor_id)
    if reason == "not_found":
        raise HTTPException(status_code=404, detail="Sponsor not found")
    if reason == "has_trials":
        raise HTTPException(status_code=409, detail="Cannot delete sponsor: trials are still linked to it")
    return sponsor
```

```
def delete_sponsor(db: Session, sponsor_id: int):
    sponsor = get_sponsor(db, sponsor_id)
    if not sponsor:
        return None, "not_found"

    # Prevent deletion if any trial still points to this sponsor
    linked_trials = db.query(models.Trial).filter(models.Trial.sponsor_id == sponsor_id).count()
    if linked_trials > 0:
        return None, "has_trials"

    db.delete(sponsor)
    db.commit()
    return sponsor, None
```

- f. DELETE – When no trials are linked deletion works
 CREATED the sponsor 2 with no trial linked

The screenshot shows a REST client interface with the following details:

- Request:** POST to `http://127.0.0.1:8425/sponsors`. The body is a JSON object: `{"sponsor_name": "Moderna Trials Inc", "contact_person": "Dr. James Lee", "email": "james.lee@moderna.com"}`.
- Response:** 200 OK, 34 ms, 307 B. The response body is a JSON object: `{"id": 2, "sponsor_name": "Moderna Trials Inc", "contact_person": "Dr. James Lee", "email": "james.lee@moderna.com", "created_at": "2026-10-02T18:51:10", "updated_at": "2026-10-02T18:51:10"}`.

Sponsor 2 deleted as it was not linked to the trial

HW4-s5825 > DELETE sponsor- When no trials are linked

DELETE http://127.0.0.1:8425/sponsors/2

Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies 1 Headers 4 Test Results 200 OK 14 ms 307 B Save Response

JSON Preview Visualize

```

1 {
2   "id": 2,
3   "sponsor_name": "Moderna Trials Inc",
4   "contact_person": "Dr. James Lee",
5   "email": "james.lee@moderna.com",
6   "created_at": "2026-10-02T18:51:10",
7   "updated_at": "2026-10-02T18:51:10"
8 }

```

g. Pagination:

There are 3 sponsors with sponsor with id=1 linked to the trial

GET http://127.0.0.1:8425/sponsors/

Docs Params Authorization Headers 7 Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies 1 Headers 4 Test Results 200 OK 7 ms 673 B Sa

JSON Preview Visualize

```

1 [
2   {
3     "id": 1,
4     "sponsor_name": "Pfizer Research",
5     "contact_person": "Dr. Sarah Chen",
6     "email": "sarah.chen@pfizer.com",
7     "created_at": "2026-10-02T18:26:38",
8     "updated_at": "2026-10-02T18:26:38"
9   },
10  {
11    "id": 3,
12    "sponsor_name": "Novartis Research",
13    "contact_person": "Dr. Emma Wilson",
14    "email": "emma.wilson@novartis.com",
15    "created_at": "2026-10-02T18:56:17",
16    "updated_at": "2026-10-02T18:56:17"
17  },
18  {
19    "id": 4,
20    "sponsor_name": "AstraZeneca Labs",
21    "contact_person": "Dr. Raj Patel",
22    "email": "raj.patel@astrazeneca.com",
23    "created_at": "2026-10-02T18:56:32",
24    "updated_at": "2026-10-02T18:56:32"
25  }
26 ]

```

When the limit is set to 1 it retrieved only one sponsor that is ONLY the first sponsor since it has the lowest id

HW4-s5825 > GET - Pagination

GET http://127.0.0.1:8425/sponsors?skip=0&limit=1

Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
✓ skip	0			
✓ limit	1			
Key	Value	Description		

Body Cookies 1 Headers 4 Test Results 200 OK • 6 ms • 307 B Save Response

{ } JSON Preview Visualize

```
1 [
2   {
3     "id": 1,
4     "sponsor_name": "Pfizer Research",
5     "contact_person": "Dr. Sarah Chen",
6     "email": "sarah.chen@pfizer.com",
7     "created_at": "2026-10-02T18:26:38",
8     "updated_at": "2026-10-02T18:26:38"
9   }
10 ]
```

The proof is that each call returns a DIFFERENT single sponsor, in order. If pagination were broken (like if skip was being ignored), all three calls would return the exact same sponsor (whichever one is first) showing that skip actually changes which record is skipped is what demonstrates it's working, not just that a result came back.

When limit = 1

HW4-s5825 > GET - Pagination

GET http://127.0.0.1:8425/sponsors?skip=0&limit=1

Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
✓ skip	0			
✓ limit	1			
Key	Value	Description		

Body Cookies 1 Headers 4 Test Results 200 OK • 7 ms • 307 B Save Response

{ } JSON Preview Visualize

```
1 [
2   {
3     "id": 1,
4     "sponsor_name": "Pfizer Research",
5     "contact_person": "Dr. Sarah Chen",
6     "email": "sarah.chen@pfizer.com",
7     "created_at": "2026-10-02T18:26:38",
8     "updated_at": "2026-10-02T18:26:38"
9   }
10 ]
```

When limit = 2

HW4-s5825 > GET - Pagination

GET http://127.0.0.1:8425/sponsors?skip=0&limit=2

Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
✓ skip	0			
✓ limit	2			
Key	Value	Description		

Body Cookies 1 Headers 4 Test Results 200 OK • 6 ms • 491 B Save Response

JSON Preview Visualize

```
1 [
2   {
3     "id": 1,
4     "sponsor_name": "Pfizer Research",
5     "contact_person": "Dr. Sarah Chen",
6     "email": "sarah.chen@pfizer.com",
7     "created_at": "2026-10-02T18:26:38",
8     "updated_at": "2026-10-02T18:26:38"
9   },
10  {
11    "id": 3,
12    "sponsor_name": "Novartis Research",
13    "contact_person": "Dr. Emma Wilson",
14    "email": "emma.wilson@novartis.com",
15    "created_at": "2026-10-02T18:56:17",
16    "updated_at": "2026-10-02T18:56:17"
17  }
18 ]
```

When limit = 3 and skip = 0

HW4-s5825 > GET - Pagination

GET http://127.0.0.1:8425/sponsors?skip=0&limit=3

Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
✓ skip	0			
✓ limit	3			
Key	Value	Description		

Body Cookies 1 Headers 4 Test Results 200 OK • 5 ms • 673 B Save Response

JSON Preview Visualize

```
1 [
2   {
3     "id": 1,
4     "sponsor_name": "Pfizer Research",
5     "contact_person": "Dr. Sarah Chen",
6     "email": "sarah.chen@pfizer.com",
7     "created_at": "2026-10-02T18:26:38",
8     "updated_at": "2026-10-02T18:26:38"
9   },
10  {
11    "id": 3,
12    "sponsor_name": "Novartis Research",
13    "contact_person": "Dr. Emma Wilson",
14    "email": "emma.wilson@novartis.com",
15    "created_at": "2026-10-02T18:56:17",
16    "updated_at": "2026-10-02T18:56:17"
17  },
18  {
19    "id": 4,
20    "sponsor_name": "AstraZeneca Labs",
21    "contact_person": "Dr. Raj Patel",
22    "email": "raj.patel@astrazeneca.com",
23    "created_at": "2026-10-02T18:56:32",
24    "updated_at": "2026-10-02T18:56:32"
25  }
26 ]
```

When limit is not selected and skip = 0 it will show all three sponsors

HW4-s5825 > GET - Pagination

GET

Docs Params Authorization Headers 7 Body Scripts Settings

Query Params

Key	Value
☒ skip	0
☐ limit	3
Key	Value

Body Cookies 1 Headers 4 Test Results 200 OK

{ } JSON Preview Visualize

```
1 [
2   {
3     "id": 1,
4     "sponsor_name": "Pfizer Research",
5     "contact_person": "Dr. Sarah Chen",
6     "email": "sarah.chen@pfizer.com",
7     "created_at": "2026-10-02T18:26:38",
8     "updated_at": "2026-10-02T18:26:38"
9   },
10  {
11    "id": 3,
12    "sponsor_name": "Novartis Research",
13    "contact_person": "Dr. Emma Wilson",
14    "email": "emma.wilson@novartis.com",
15    "created_at": "2026-10-02T18:56:17",
16    "updated_at": "2026-10-02T18:56:17"
17  },
18  {
19    "id": 4,
20    "sponsor_name": "AstraZeneca Labs",
21    "contact_person": "Dr. Raj Patel",
22    "email": "raj.patel@astrazeneca.com",
23    "created_at": "2026-10-02T18:56:32",
24    "updated_at": "2026-10-02T18:56:32"
25  }
26 ]
```

When skip = 2 it will skip first two id's and will show the 3rd one

HW4-s5825 > GET - Pagination Save Share

GET Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
☒ skip	2			
☐ limit	3			
Key	Value	Description		

Body Cookies 1 Headers 4 Test Results 200 OK 6 ms 311 B Save Response

{ } JSON Preview Visualize

```
1 [
2   {
3     "id": 4,
4     "sponsor_name": "AstraZeneca Labs",
5     "contact_person": "Dr. Raj Patel",
6     "email": "raj.patel@astrazeneca.com",
7     "created_at": "2026-10-02T18:56:32",
8     "updated_at": "2026-10-02T18:56:32"
9   }
10 ]
```

Code:

```
@app.get("/sponsors", response_model=list[Schema.SponsorOut])
def list_sponsors(skip: int = 0, limit: int = 20, db: Session = Depends(get_db),
                  _s=Depends(require_session)):
    return crud.get_sponsors(db, skip, limit)

def get_sponsors(db: Session, skip: int = 0, limit: int = 20):
    return db.query(models.Sponsor).order_by(models.Sponsor.id.asc()).offset(skip).limit(limit).all()
```

- h. POST: Duplicate email: Post sponsors again with same email id which will give 409
Email already exists error message

HTTP HW4-s5825 > POST - Create sponsor

POST http://127.0.0.1:8425/sponsors

Body

```
1 { "sponsor_name": "AstraZeneca Labs", "contact_person": "Dr. Raj Patel", "email": "sarah.chen@pfizer.com" }
```

409 Conflict • 6 ms • 166 B

Resolve 409 Conflict error

JSON

```
1 {
2   "detail": "Email already exists"
3 }
```

```
sponsors_crud (add need a valid session)
@app.post("/sponsors", response_model=Schema.SponsorOut)
def add_sponsor(payload: Schema.SponsorCreate, db: Session = Depends(get_db),
               _s=Depends(require_session)):
    try:
        return crud.create_sponsor(db, payload)
    except IntegrityError:
        db.rollback()
        raise HTTPException(status_code=409, detail="Email already exists")
```

- i. GET: Search for sponsor id that doesnot exists gives 404 Not found error with error message as Sponsor not found:

HW4-s5825 > GET:Find ID that doesn't exist

GET http://127.0.0.1:8425/sponsors/999

Send

Docs Params Authorization Headers 7 Body Scripts Settings

Query Params

Key	Value	Description	Bulk Ed
Key	Value	Description	

Body Cookies 1 Headers 4 Test Results

404 Not Found 7 ms 164 B

Find a valid sponsor ID

Diagnose Decline

JSON Preview Debug with AI

```

1 {
2   "detail": "Sponsor not found"
3 }

```

```

@app.get("/sponsors/{sponsor_id}", response_model=Schema.SponsorOut)
def get_sponsor(sponsor_id: int, db: Session = Depends(get_db),
                _s=Depends(require_session)):
    sponsor = crud.get_sponsor(db, sponsor_id)
    if not sponsor:
        raise HTTPException(status_code=404, detail="Sponsor not found")
    return sponsor

```

```

def get_sponsor(db: Session, sponsor_id: int):
    return db.query(models.Sponsor).filter(models.Sponsor.id == sponsor_id).first()

```

The same error checks when there is an update(PUT) for non-existing sponsor id, It returns 404 Not found error.

Checked in database for the sponsor id which are not null

```

SELECT * FROM s5825_rel.sponsors;
SELECT id, trial_title, available_slots, sponsor_id FROM s5825_rel.trials WHERE sponsor_id IS NOT NULL;

```

```

● rutujaste@Rutujaste-MacBook-Pro data260-5825 % source /Users/rutujaste/Documents/data260-5825/venv/bin/activate
● (venv) rutujaste@Rutujaste-MacBook-Pro data260-5825 % mysql -u root -p -e "SELECT * FROM s5825_rel.sponsors; SELECT id, trial_title, available_slots, sponsor_id FROM s5825_rel.trials WHERE sponsor_id IS NOT NULL;"
Enter password:
+----+-----+-----+-----+-----+-----+
| id | sponsor_name | contact_person | email | created_at | updated_at |
+----+-----+-----+-----+-----+-----+
| 1 | Pfizer Research | Dr. Sarah Chen | sarah.chen@pfizer.com | 2026-10-02 18:26:38 | 2026-10-02 18:26:38 |
| 3 | Novartis Research | Dr. Emma Wilson | emma.wilson@novartis.com | 2026-10-02 18:56:17 | 2026-10-02 18:56:17 |
| 4 | AstraZeneca Labs | Dr. Raj Patel | raj.patel@astrazeneca.com | 2026-10-02 18:56:32 | 2026-10-02 18:56:32 |
+----+-----+-----+-----+-----+-----+
+----+-----+-----+-----+
| id | trial_title | available_slots | sponsor_id |
+----+-----+-----+-----+
| 5001 | Pfizer Diabetes Phase III | 25 | 1 |
+----+-----+-----+-----+
○ (venv) rutujaste@Rutujaste-MacBook-Pro data260-5825 %

```

III : Redux Client

a. Home page showing list of trials

Clinical Trial Registry

Home Add Trial Logout

Clinical Trials + Add Trial

ID	TRIAL TITLE	NCT NUMBER	SLOTS	SPONSOR	ACTIONS
1	Sitagliptin for Stroke Prevention #1	NCT10000000	10	-	Edit Delete
2	Metformin for Kidney Disease #2	NCT10000001	10	-	Edit Delete
3	Metformin for Kidney Disease #3	NCT10000002	10	-	Edit Delete
4	Aspirin for Hypertension #4	NCT10000003	10	-	Edit Delete
5	Sitagliptin for Obesity #5	NCT10000004	10	-	Edit Delete
6	Empagliflozin for Type 2 Diabetes #6	NCT10000005	10	-	Edit Delete
7	Lisinopril for Kidney Disease #7	NCT10000006	10	-	Edit Delete
8	Aspirin for Heart Failure #8	NCT10000007	10	-	Edit Delete
9	Empagliflozin for Kidney Disease #9	NCT10000008	10	-	Edit Delete
10	Empagliflozin for High Cholesterol #10	NCT10000009	10	-	Edit Delete
11	Metformin for Stroke Prevention #11	NCT10000010	10	-	Edit Delete
12	Sitagliptin for High Cholesterol #12	NCT10000011	10	-	Edit Delete

```
.addCase(fetchTrials.pending, (state) => { state.status = "loading"; state.error = null; })
.addCase(fetchTrials.fulfilled, (state, action) => { state.status = "succeeded"; state.items = action.payload; })
.addCase(fetchTrials.rejected, (state, action) => { state.status = "failed"; state.error = action.payload; })

export const fetchTrials = createAsyncThunk(
  "trials/fetch",
  async (_, { rejectWithValue }) => {
    try {
      const res = await api.get("/trials");
      return res.data;
    } catch (err) {
      return rejectWithValue(err.response?.data?.detail || "Failed to load trials");
    }
  }
);
```

b. Home showing the new trial after creating one with the linked sponsor.

Clinical Trial Registry

Home Add Trial Logout

Add Clinical Trial

Trial Title

Redux Demo Trial

NCT Number

NCT88888001

Available Slots

15

Sponsor ID (optional)

1

Add Trial Cancel

New trial comes to the top of the list

localhost:5173

Clinical Trial Registry						
			Home		Add Trial	Logout

Clinical Trials						
						+ Add Trial
ID	TRIAL TITLE	NCT NUMBER	SLOTS	SPONSOR	ACTIONS	
5002	Redux Demo Trial	NCT88888001	15	1	Edit	Delete
5001	Pfizer Diabetes Phase III	NCT99999001	25	1	Edit	Delete
5000	Empagliflozin for Obesity #5000	NCT10004999	10	-	Edit	Delete
4999	Aspirin for Type 2 Diabetes #4999	NCT10004998	10	-	Edit	Delete
4998	Aspirin for Kidney Disease #4998	NCT10004997	10	-	Edit	Delete

```
export const createTrial = createAsyncThunk(
  "trials/create",
  async (trial, { rejectWithValue }) => {
    try {
      const res = await api.post("/trials", trial);
      return res.data;
    } catch (err) {
      return rejectWithValue(err.response?.data?.detail || "Failed to create trial");
    }
  }
);

.addCase(createTrial.fulfilled, (state, action) => { state.items.push(action.payload); })
.addCase(createTrial.rejected, (state, action) => { state.error = action.payload; })

{[...trials].reverse().slice(0, 50).map((t) => {
  <tr key={t.id}>
    <td>{t.id}</td>
    <td>{t.trial_title}</td>
    <td><span className="badge">{t.nct_number}</span></td>
    <td>{t.available_slots}</td>
    <td>{t.sponsor_id ?? "-"}</td>
    <td>
      <div className="actions">
```

- c. Update trial: Updated the trial with new name and slots = 30 and it gets updated in trial list

```
export const updateTrial = createAsyncThunk(
  "trials/update",
  async ({ id, data }, { rejectWithValue }) => {
    try {
      const res = await api.put(`/trials/${id}`, data);
      return res.data;
    } catch (err) {
      return rejectWithValue(err.response?.data?.detail || "Failed to update trial");
    }
  }
);

.addCase(updateTrial.fulfilled, (state, action) => {
  const i = state.items.findIndex((t) => t.id === action.payload.id);
  if (i !== -1) state.items[i] = action.payload;
})
.addCase(updateTrial.rejected, (state, action) => { state.error = action.payload; })
```

localhost:5173/update/5002

Clinical Trial Registry Home Add Trial Logout

Update Trial #5002

Trial Title

NCT Number

Available Slots

Sponsor ID (optional)

Update Trial Cancel

localhost:5173

Clinical Trial Registry Home Add Trial Logout

Clinical Trials

+ Add Trial

ID	TRIAL TITLE	NCT NUMBER	SLOTS	SPONSOR	ACTIONS
5002	Redux Demo Trial UPDATED	NCT88888001	30	1	Edit Delete

- d. DELETE: Delete the trial from trial list: Clicked on delete button and the list gets updated.

ID	TRIAL TITLE	NCT NUMBER	SLOTS	SPONSOR	ACTIONS
5001	Pfizer Diabetes Phase III	NCT99999001	25	1	Edit Delete

localhost:5173

Clinical Trial Registry Home Add Trial Logout

Clinical Trials

+ Add Trial

ID	TRIAL TITLE	NCT NUMBER	SLOTS	SPONSOR	ACTIONS
5000	Empagliflozin for Obesity #5000	NCT10004999	10	-	Edit Delete
4999	Aspirin for Type 2 Diabetes #4999	NCT10004998	10	-	Edit Delete
4998	Aspirin for Kidney Disease #4998	NCT10004997	10	-	Edit Delete
4997	Aspirin for Kidney Disease #4997	NCT10004996	10	-	Edit Delete

```
export const deleteTrial = createAsyncThunk(
  "trials/delete",
  async (id, { rejectWithValue }) => {
    try {
      await api.delete(`/trials/${id}`);
      return id;
    } catch (err) {
      return rejectWithValue(err.response?.data?.detail || "Failed to delete trial");
    }
  }
)
```

```
.addCase(deleteTrial.fulfilled, (state, action) => {
  state.items = state.items.filter((t) => t.id !== action.payload);
})
.addCase(deleteTrial.rejected, (state, action) => { state.error = action.payload; });
```

Part 2A: the Meals MCP server:

Here the MCP_Inspector is the test client that connects to my meals server and uses the server tools to do following operations:

Started MCP Inspector (Client)

```
(venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % cd ~/Documents/data260-5825
source venv/bin/activate
npx @modelcontextprotocol/inspector python code/mcp/meals_server.py

Starting MCP inspector...

MCP Inspector Web is up and running at:
http://127.0.0.1:6274?MCP_INSPECTOR_API_TOKEN=fe0f689d0cb1be980cc1a6fe4935b875634d6114d15a7a1506a243627950e31f

Sandbox (MCP Apps): http://127.0.0.1:6275/sandbox

Auth token: fe0f689d0cb1be980cc1a6fe4935b875634d6114d15a7a1506a243627950e31f

Secrets: OS keychain

Opening browser...
```

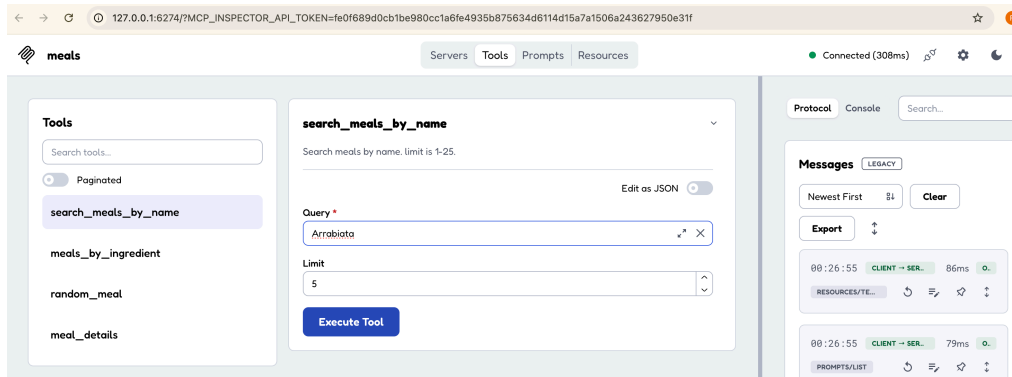
Logging rule:

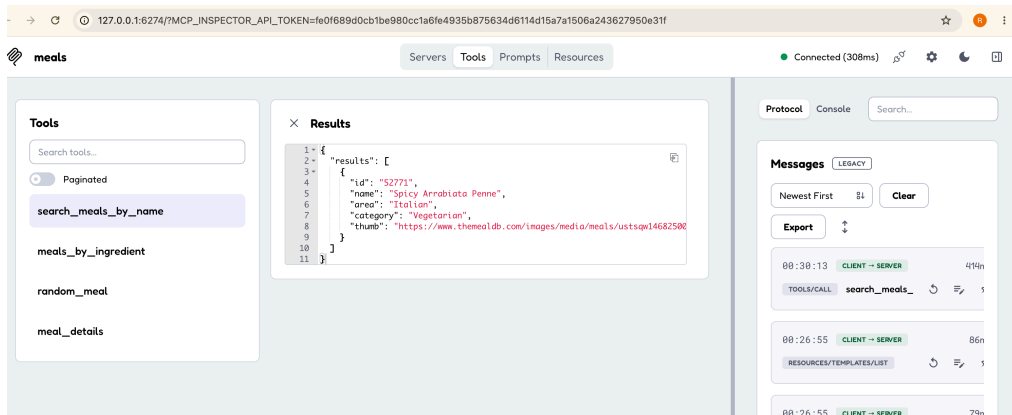
```
# STDIO servers must NEVER print to stdout, so all logs go to stderr
logging.basicConfig(stream=sys.stderr, level=logging.INFO)
log = logging.getLogger("meals")
```

Network or JSON errors are caught and raised as a clean error message, which the Inspector displays.

```
except httpx.HTTPError as e:
    log.error("network error: %s", e)
    raise RuntimeError(f"TheMealDB request failed: {e}")
except ValueError as e:
    raise RuntimeError(f"TheMealDB returned invalid JSON: {e}")
```

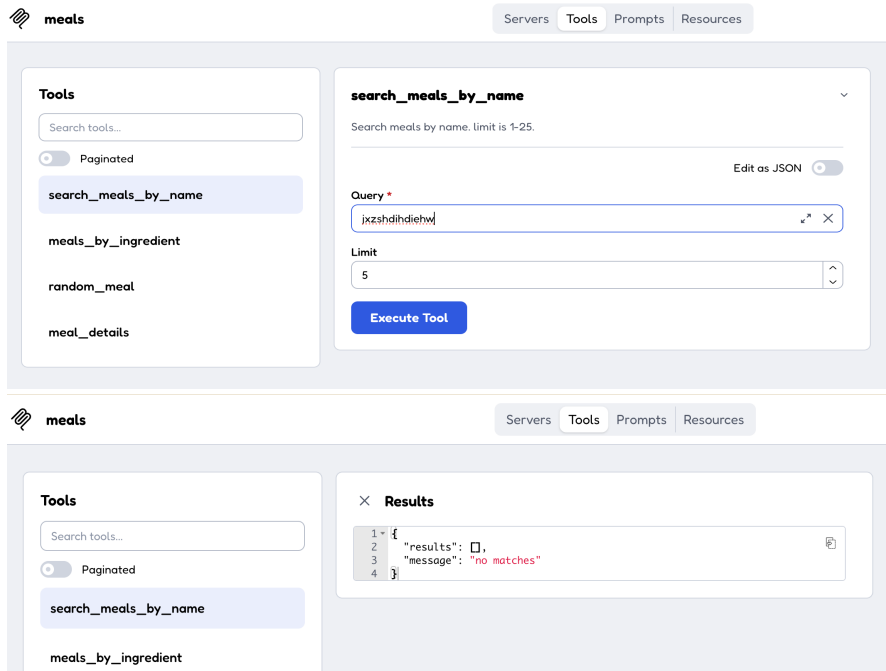
searched the meal by name:



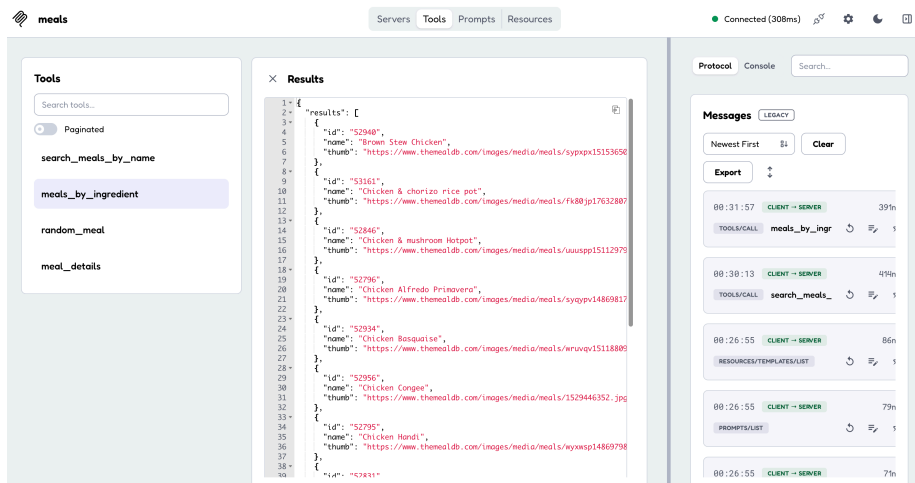
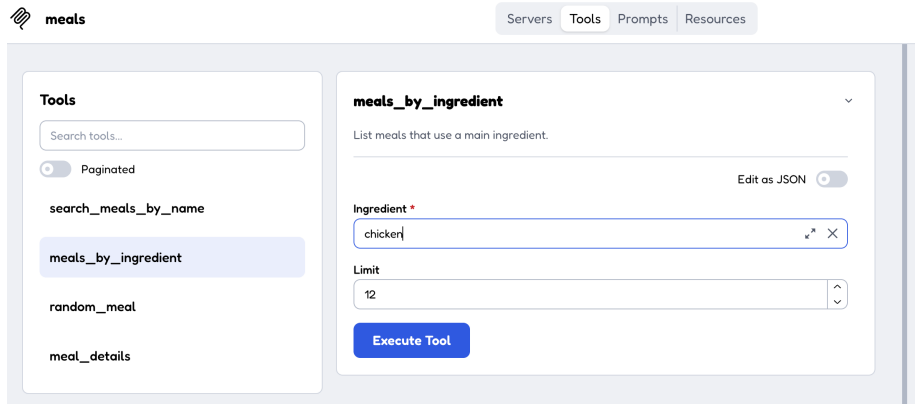


```
@mcp.tool()
def search_meals_by_name(query: str, limit: int = 5) -> dict:
    """Search meals by name. limit is 1-25."""
    limit = max(1, min(25, limit))
    meals = _get("search.php", {"s": query}).get("meals")
    if not meals:
        return {"results": [], "message": "no matches"}
    return {"results": [
        {"id": m["idMeal"], "name": m["strMeal"], "area": m["strArea"],
         "category": m["strCategory"], "thumb": m["strMealThumb"]}
        for m in meals[:limit]]}
```

IF added random name it provides message as no matches in json format

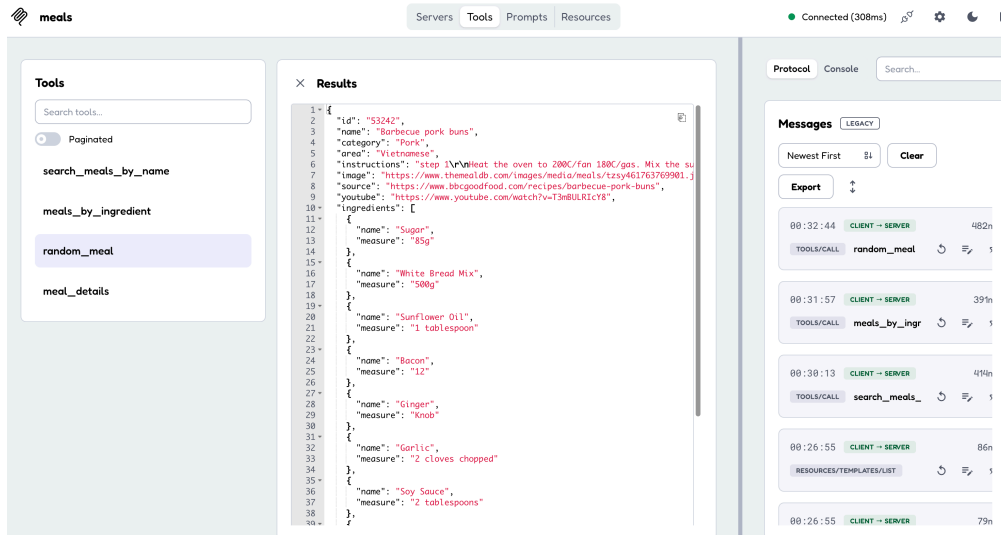


Searched with meals by ingredient and limit=12

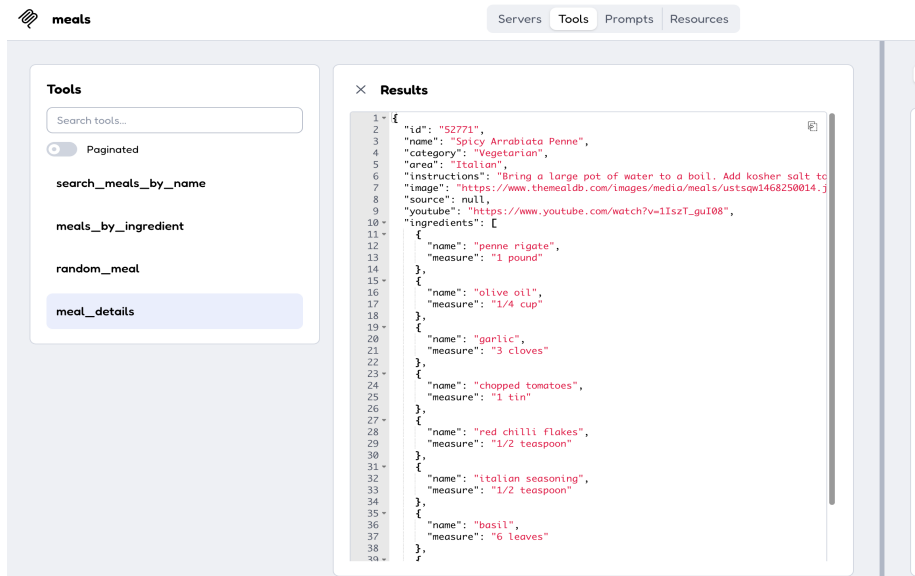
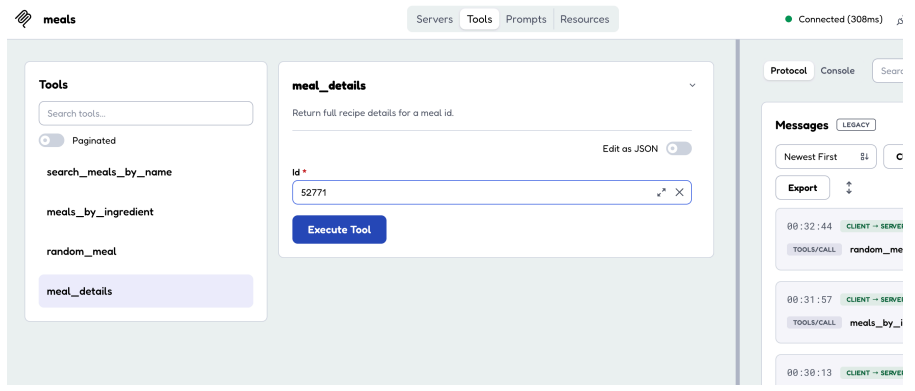


```
@mcp.tool()
def meals_by_ingredient(ingredient: str, limit: int = 12) -> dict:
    """List meals that use a main ingredient."""
    limit = max(1, min(50, limit))
    meals = _get("filter.php", {"i": ingredient}).get("meals")
    if not meals:
        return {"results": [], "message": "no matches"}
    return {"results": [
        {"id": m["idMeal"], "name": m["strMeal"], "thumb": m["strMealThumb"]}
        for m in meals[:limit]]
    ]}
```

Below tool returns one random meal without any input



Searching meal based on meal id that provides the meal details



Code:

```
mcp = FastMCP("meals")
BASE = "https://www.themealdb.com/api/json/v1/1"
```

Part 2B: Domain MCP Server (required):

```
(venv) rutujaste@Rutujas-MacBook-Pro data260-5825 % cd ~/Documents/data260-5825
source venv/bin/activate
npx @modelcontextprotocol/inspector python code/mcp/trials_server.py
Starting MCP inspector...

MCP Inspector Web is up and running at:
http://127.0.0.1:6274?MCP_INSPECTOR_API_TOKEN=b5428b8c6920a6b21b943c677421d5f2e14c4e535d4bfa8a7f3aa2fa1e3a9e8f

Sandbox (MCP Apps): http://127.0.0.1:6275/sandbox

Auth token: b5428b8c6920a6b21b943c677421d5f2e14c4e535d4bfa8a7f3aa2fa1e3a9e8f

Secrets: OS keychain

Opening browser...
```

The screenshot displays the MCP Inspector Web interface in a browser. The address bar shows the URL: `127.0.0.1:6274?MCP_INSPECTOR_API_TOKEN=b5428b8c6920a6b21b943c677421d5f2e14c4e535d4bfa8a7f3aa2fa1e3a9e8f`. The interface has a top navigation bar with tabs for Servers, Tools, Prompts, and Resources. A status bar at the top right indicates 'Connected (441ms)' along with settings and theme icons.

The main content area is divided into two panels. The left panel, titled 'Servers', shows a 'Read-only session' warning and a list of servers. One server, named 'python', is shown as 'Connected' with a toggle switch. Below the server name, it indicates 'STDIO Standard I/O' and 'MCP 2025-11-25'. A code editor displays the command `python code/mcp/trials_server.py`. An 'Export' button is visible in the top right of the server list.

The right panel, titled 'Messages', shows a 'LEGACY' tab and a 'Newest First' sort order. It contains a list of messages with timestamps and durations. The first message is at 00:58:06, showing a 'CLIENT -> SER...' interaction with a duration of 105ms. The second message is also at 00:58:06, showing a 'CLIENT -> SER...' interaction with a duration of 96ms. The third message is at 00:58:06, showing a 'CLIENT -> SER...' interaction with a duration of 83ms. Each message entry includes a 'Resources/TE...' button and a 'Prompts/List' button.

Tools :

Search trials will provide the complete information :

→ ↻ 127.0.0.1:6274/?MCP_INSPECTOR_API_TOKEN=7494f7c0e4ecb0d4377399dac3005c2a94418a7dbb4f17456893a8f9c367df96

Continue where you left off: Chrome can restore your tabs every time you restart. To turn this on, go to [Settings](#).

trials

ServersToolsPromptsResources

Tools

Search tools...

Paginated

search_trials

get_trial

count_trials_by_sponsor

search_trials

Search clinical trials by title. limit is 1-50.

Edit as JSON

Query *
Diabetes ✓ ✕

Limit
10 ⬆ ⬆

Execute Tool

trials

ServersToolsPromptsResources

Cor

Tools

Search tools...

Paginated

search_trials

get_trial

count_trials_by_sponsor

Results

```
1- {
2   "ok": true,
3   "data": [
4     {
5       "id": 6,
6       "trial_title": "Empagliflozin for Type 2 Diabetes #6",
7       "nct_number": "NCT10000005",
8       "available_slots": 10,
9       "sponsor_id": null
10    },
11    {
12      "id": 10,
13      "trial_title": "Empagliflozin for Type 2 Diabetes #10",
14      "nct_number": "NCT10000017",
15      "available_slots": 10,
16      "sponsor_id": null
17    },
18    {
19      "id": 25,
20      "trial_title": "Metformin for Type 2 Diabetes #25",
21      "nct_number": "NCT10000024",
22      "available_slots": 10,
23      "sponsor_id": null
24    },
25    {
26      "id": 35,
27      "trial_title": "Aspirin for Type 2 Diabetes #35",
28      "nct_number": "NCT10000034",
29      "available_slots": 10,
30      "sponsor_id": null
31    },
32    {
33      "id": 39,
34      "trial_title": "Lisinopril for Type 2 Diabetes #39",
35      "nct_number": "NCT10000030",
36      "available_slots": 10,
37      "sponsor_id": null
38    },
39    {
40      "id": 42,
41      "trial_title": "Lisinopril for Type 2 Diabetes #42",
42      "nct_number": "NCT10000041",
43      "available_slots": 10,
44      "sponsor_id": null
45    },
46    {
47      "id": 43,
48      "trial_title": "Semaglutide for Type 2 Diabetes #43",
49      "nct_number": "NCT10000042",
50      "available_slots": 10,
51      "sponsor_id": null
52    }
53  ]
54 }
```

Protocol

Messa

Newest

Expo

17:3
TOOL

17:2
TOOL

17:2
RESO

17:2
PROP

17:2
TOOL

17:0

v2.9.0

Copyright ©

With invalid input:

search_trials

Search clinical trials by title. limit is 1-50.

Edit as JSON

Query

Limit

5

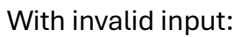
Execute Tool

×

Results

```
1 {
2   "ok": false,
3   "data": null,
4   "error": "query must be a non-empty string"
5 }
```

Get Trials will pull the trial by adding the trial id



Count trials by sponsor id will give the aggregate count of trials and total available slots for sponsor

Count trials by sponsor id will give the aggregate count of trials and total available slots for sponsor

trials

ServersToolsPromptsResources

Tools

Search tools...

Paginated

search_trials

get_trial

count_trials_by_sponsor

count_trials_by_sponsor

Aggregate: number of trials and total available slots for a sponsor.

Edit as JSON

Sponsor Id *
1

Execute Tool

×

Results

```
1 {  
2   "ok": true,  
3   "data": {  
4     "sponsor_id": 1,  
5     "sponsor_name": "Pfizer Research",  
6     "trial_count": 1,  
7     "total_available_slots": 25  
8   },  
9   "error": null  
10 }
```

With invalid input

count_trials_by_sponsor

Aggregate: number of trials and total available slots for a sponsor.

Sponsor Id *

5001

Execute Tool

× Results

```
1 {  
2   "ok": false,  
3   "data": null,  
4   "error": "sponsor 5001 not found"  
5 }
```

Part 3 - Tool Contracts Under Stress:

- a. **Rejected calls from Part 2B:** All three tools return the same envelope, {ok, data, error}. Each rejected call below is the same one captured in the Part 2B Inspector screenshots.

The first two calls are rejected by input validation (the input is malformed). The third passes validation and is rejected because the record does not exist. In all three cases the tool returns an error instead of raising an exception.

Tool	Expected input schema	Rejected JSON input	Returned error output	Why it was rejected
search_trials	{"query": string (non-empty), "limit": integer 1-50}	{"query": "", "limit": 5}	{"ok": false, "data": null, "error": "query must be a non-empty string"}	The query is empty, so there is nothing to search for. The input check fails before any database call.
get_trial	{"trial_id": integer >= 1}	{"trial_id": 0}	{"ok": false, "data": null, "error": "trial_id must be a positive integer"}	Ids start at 1, so 0 can never match a trial.
count_trials_by_sponsor	{"sponsor_id": integer >= 1}	{"sponsor_id": 5001}	{ "ok": false, "data": null, "error": "sponsor 5001 not found" }	The input has the right shape, but no sponsor with id 5001 exists. This is a lookup failure, not a format failure.

b. Timeout and bounded exponential-backoff retry:

```

MAX_RETRIES = 3           # retries after the first attempt (4 attempts total)
BASE_DELAY = 0.05         # seconds; doubles each retry: 0.05, 0.1, 0.2
TIMEOUT = 2.0             # seconds per attempt

```

```

try:
    for attempt in range(max_retries + 1):
        attempts = attempt + 1
        try:
            future = pool.submit(fn, *args, **kwargs)
            result = future.result(timeout=timeout)
            return result, attempts
        except FutureTimeout:
            last_error = f"timed out after {timeout}s"
        except TransientError as e:
            last_error = str(e)
        except Exception as e:
            last_error = f"unexpected error: {e}"
        log.warning("attempt %d failed: %s", attempts, last_error)
        if attempt < max_retries:
            time.sleep(base_delay * (2 ** attempt))
    return err(f"failed after {attempts} attempts: {last_error}"), attempts

```

Each attempt has a 2-second timeout. A failed or timed-out attempt is retried up to 3 more times, waiting 0.05s, 0.1s, then 0.2s (the delay doubles each time). The retry count is bounded, so the call can never loop forever. If every attempt fails, the function returns `errpr` instead of raising an exception.

```

(venv) PS C:\Users\vivek\data260-5825\code\mcp> python part3_experiment.py 2>$null | Tee-Object
hw05\RUN_LOG_part3.txt
VERIFY_SEED = 265825

=== DEMO: success on the first attempt ===
outcomes script : [False] (True = injected failure)
attempts used    : 1
latency (ms)     : 29.8
result ok        : True
error            : None

=== DEMO: failure first, success after retry ===
outcomes script : [True, False] (True = injected failure)
attempts used    : 2
latency (ms)     : 54.4
result ok        : True
error            : None

=== DEMO: failure after all retries (clean error) ===
outcomes script : [True, True, True, True] (True = injected failure)
attempts used    : 4
latency (ms)     : 352.7
result ok        : False
error            : failed after 4 attempts: injected failure

| Injected failure rate | Success rate | Mean latency (ms) | p99 latency (ms) |
|---|---|---|---|
| 0% | 100.0% | 0.81 | 1.66 |
| 20% | 100.0% | 34.82 | 352.89 |
| 50% | 94.0% | 103.72 | 356.3 |

rows written: 150 (expected 150)

```

- Success on the first attempt. - Latency 29.8 ms with 1 attempt and result is true
- Failure on the first attempt followed by success after a retry- Latency 54.4 ms with 2 attempts and result is true
- Failure after all allowed retries, returning a clean error result instead of crashing- Latency 352.7 ms with 4 attempts and result is false as it failed.

These are from an earlier run; the repeat run in section c gives slightly different latencies because timing varies, while the success/failure sequence is identical.

c. Reproducible fault injection (VERIFY_SEED):

Each attempt draws one random number, and the attempt fails if the number is below the failure rate (0%, 20%, 50%).

```
class FlakyInjector:
    """Seeded failure injector. Same seed gives the same fail/succeed sequence."""

    def __init__(self, rate, seed):
        self.rate = rate
        self.rng = random.Random(seed)

    def wrap(self, fn):
        def inner(*args, **kwargs):
            if self.rng.random() < self.rate:
                raise TransientError("injected failure")
            return fn(*args, **kwargs)
        return inner

Rutuja Saste> cd code\mcp
Rutuja Saste> python part3_experiment.py 2>$null | Tee-Object -FilePath ../../reports/hw05\RUN_LOG_part3_run2.txt
VERIFY_SEED = 265825

=== DEMO: success on the first attempt ===
outcomes script : [False] (True = injected failure)
attempts used    : 1
latency (ms)     : 37.5
result ok        : True
error            : None

=== DEMO: failure first, success after retry ===
outcomes script : [True, False] (True = injected failure)
attempts used    : 2
latency (ms)     : 54.5
result ok        : True
error            : None

=== DEMO: failure after all retries (clean error) ===
outcomes script : [True, True, True, True] (True = injected failure)
attempts used    : 4
latency (ms)     : 352.8
result ok        : False
error            : failed after 4 attempts: injected failure

| Injected failure rate | Success rate | Mean latency (ms) | p99 latency (ms) |
|---|---|---|---|
| 0% | 100.0% | 0.99 | 3.1 |
| 20% | 100.0% | 35.4 | 355.13 |
| 50% | 94.0% | 103.93 | 355.86 |

rows written: 150 (expected 150)
```

The three retry demos and the 150-call fault-injection run, using VERIFY_SEED = 265825. Each call used a 2 s timeout and up to 3 retries with backoff of 0.05 s, 0.1 s and 0.2 s. All 150 calls are saved in reports/hw05/raw/fault_injection_150.csv.

Demo	Attempts used	Latency	Result
Success on the first attempt	1	37.5 ms	ok: True
Failure first, then success after a retry	2	54.5 ms	ok: True
Failure after all retries	4	352.8 ms	ok: False, clean error, no crash

Injected failure rate	Succesrate	Mean latency (ms)	p99 latency (ms)
0%	100.0%	0.99	3.10
20%	100.0%	35.40	355.13
50%	94.0%	103.93	355.86

From the results we can say that the retry policy suitable for an interactive assistant mostly as, with no injected failures (0%), calls finish in about 1 ms. With 20% failures, retrying recovered every call (100% success) and the mean latency was only 35.40 ms. The slowest calls (p99, about 355 ms) are the cost of waiting between retries: $0.05 + 0.1 + 0.2 = 0.35$ seconds. That is under half a second, so a user would barely notice. With 50% failures the policy is weaker. Success was 94%, so 3 of 50 calls still failed after all 4 attempts, and the mean latency rose to 103.93 ms because many calls needed retries. Even so, those failures came back as a clean `{ok: false}` error and nothing crashed. If a real service failed half the time, the right fix would be to repair the service, not to add more retries.

How you would modify the number of retries, timeout, or backoff delays for batch processing, where longer processing times are acceptable?

Batch jobs can wait longer, so success matters more than speed:

Retries: raise from 3 to about 8, so almost no call fails permanently.

Timeout: raise from 2 seconds to 10 to 30 seconds, so slow but healthy calls are not cut off.

Backoff: start with a longer wait (0.5 to 1 second), double it each time up to a cap of about 30 seconds, and add a small random delay so many jobs do not retry at the same moment.

Part 4 - One Safe Tool Entry Point and Offline Tests:

9 tests across the three domain tools (valid input, invalid input, unknown tool, bad arguments, envelope shape). They run offline against a fake database, with no MySQL, no network and no LLM. Result: 9/9 PASS. `execute_tool` is the single entry point for every tool call. It looks up the tool by name, validates the inputs, and always returns a JSON string in the `{ok, data, error}` shape without raising.

```
Rutuja Saste> python test_runner.py
PASS t_search_valid
PASS t_search_empty_query
PASS t_search_bad_limit
PASS t_get_valid
PASS t_get_invalid_id
PASS t_count_invalid_sponsor_type
PASS t_unknown_tool
PASS t_bad_arguments
PASS t_envelope_shape

9/9 tests passed
Rutuja Saste> █
```

Part 5 - Agent Loop, Safety Rule, and Reflection:

I. Safety rule:

```

Rutuja Sastee python > import executor; print(executor.execute_trial_search_trials, {'query': 'Diabetes', 'limit': 1000})
{"ok": true, "data": [{"id": 6, "trial_title": "Empagliflozin for Type 2 Diabetes #6", "net_number": "NCT01888800", "available_slots": 10, "sponsor_id": null, "id": 10, "trial_title": "Empagliflozin f
or Type 2 Diabetes #10", "net_number": "NCT19000017", "available_slots": 10, "sponsor_id": null, "id": 25, "trial_title": "Metformin for Type 2 Diabetes #25", "net_number": "NCT19000024", "available
slots": 10, "sponsor_id": null, "id": 35, "trial_title": "Aspirin for Type 2 Diabetes #35", "net_number": "NCT19000034", "available_slots": 10, "sponsor_id": null, "id": 39, "trial_title": "Nitrogli
piper for Type 2 Diabetes #39", "net_number": "NCT19000038", "available_slots": 10, "sponsor_id": null, "error": null}]}
{"ok": false, "data": null, "error": "safety_nets search_trials limit 100 exceeds the maximum of 25 records per call"}
Rutuja Sastee

```

```

9 MAX_AGENT_LIMIT = 25 # safety rule: an agent may not pull more than 25 records
1
2
3 def check_safety(name, inputs):
4     """Return an error message if the call breaks the safety rule, else None."""
5     if name == "search_trials" and isinstance(inputs, dict):
6         limit = inputs.get("limit", 10)
7         if isinstance(limit, int) and limit > MAX_AGENT_LIMIT:
8             return (f"safety rule: search_trials limit {limit} exceeds "
9                     f"the maximum of {MAX_AGENT_LIMIT} records per call")
10    return None

```

The allowed call (limit 5) returns data. The blocked call (limit 100) returns {ok: false, data: null, error: "safety rule: ..."} without raising an exception.

II. Agent loop

[illegible]

```

def run_agent(user_input, model=None, db=None, max_steps=MAX_STEPS, log_path=LOG_FILE):
    model = model or OllamaModel()
    messages = [
        ("role": "system", "content": "You answer questions about clinical trials. "
        "Use the tools to get real data. Answer briefly."),
        ("role": "user", "content": user_input),
    ]
    tool_calls_made = 0
    blocked = False
    run_id = int(time.time() * 1000)
    for step in range(1, max_steps + 1):
        resp = model.chat(messages)
        msg = _get(resp, "message")
        calls = _get(msg, "tool_calls") or []
        content = _get(msg, "content") or ""
        if not calls:
            log(("run": run_id, "step": step, "event": "final",
            "answer": content, "stop_reason": "completed"), log_path)
            return {"answer": content, "steps": step, "stop_reason": "completed",
            "tool_calls": tool_calls_made, "safety_blocked": blocked}
        messages.append(("role": "assistant", "content": content))
        for c in calls:
            fn = _get(c, "function")
            name = _get(fn, "name")
            args = dict(_get(fn, "arguments") or {})
            result = executor.execute_tool(name, args, db=db)
            tool_calls_made += 1
            if "safety_rule" in result:
                blocked = True
            log(("run": run_id, "step": step, "event": "tool call", "tool": name,
            "input": args, "result": json.loads(result)), log_path)
            messages.append(("role": "tool", "content": result, "name": name))
    log(("run": run_id, "step": max_steps, "event": "stop",
    "stop_reason": "max_steps"), log_path)

```

The loop counts steps, stops at max_steps (5), and logs every tool call, input, result and the final stop reason to agent_runs.jsonl

III. Additional tests

Part 4 and Part 5 offline test runner: 11 tests, all PASS. They run without a database, network or LLM.

```

Rutuja Saste> python test_runner.py
PASS t_search_valid
PASS t_search_empty_query
PASS t_search_bad_limit
PASS t_get_valid
PASS t_get_invalid_id
PASS t_count_invalid_sponsor_type
PASS t_unknown_tool
PASS t_bad_arguments
PASS t_envelope_shape
PASS t_safety_blocks
PASS t_agent_max_steps

11/11 tests passed
Rutuja Saste>

```

Code:

```

def execute_tool(name, inputs, db=None):
    try:
        if name not in TOOLS:
            return json.dumps(core.err(f"unknown tool: {name}"))
        if not isinstance(inputs, dict):
            return json.dumps(core.err("inputs must be an object"))
        violation = check_safety(name, inputs)
        if violation:
            return json.dumps(core.err(violation))
        result = TOOLS[name](**inputs, db=db)
        return json.dumps(result)
    except TypeError as e:
        return json.dumps(core.err(f"bad arguments: {e}"))
    except Exception as e:
        return json.dumps(core.err(f"internal error: {e}"))

```

IV. Metrics:

```

INFO:httpx:HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
{"question": "How many trials does sponsor 1 have, and how many slots in total?", "steps": 2, "stop_reason": "completed", "tool_calls": 1, "safety_blocked": false}
INFO:httpx:HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
{"question": "Find trials about Diabetes and tell me the first one's NCT number.", "steps": 2, "stop_reason": "completed", "tool_calls": 1, "safety_blocked": false}
INFO:httpx:HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
{"question": "Look up trial 5001 and name its sponsor.", "steps": 2, "stop_reason": "completed", "tool_calls": 1, "safety_blocked": false}
INFO:httpx:HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"
{"question": "Give me 100 trials about Metformin.", "steps": 4, "stop_reason": "completed", "tool_calls": 3, "safety_blocked": true}

| Scenario | Steps | Stop reason | Tool calls | Safety blocked |
|---|---|---|---|---|
| How many trials does sponsor 1 have, and how many slots in total? | 2 | completed | 1 | False |
| Find trials about Diabetes and tell me the first one's NCT number. | 2 | completed | 1 | False |
| Look up trial 5001 and name its sponsor. | 2 | completed | 1 | False |
| Give me 100 trials about Metformin. | 4 | completed | 3 | True |
Rutuja Saste>

```

Reflection:

I chose scenario 4, "Give me 100 trials about Metformin," because it shows the safety rule working inside the agent loop (run 1791225521181).

Step 1: the model asked for search_trials with limit: 100. The harness sent this through execute_tool, and the safety rule (maximum 25 records per call) blocked it. It returned {ok: false, data: null, error: "safety rule: ..."} without raising an exception.

Step 2: the model read the error and retried with limit: 25. This call was allowed and returned 25 Metformin trials.

Step 3: the model repeated the same 25-record call with the arguments in a different order. This was redundant. It did not need new data, but the harness has no rule against repeated calls, so it ran and counted as a third tool call.

Step 4: the model gave its final answer, so the harness stopped with reason completed. It did not stop because of max_steps, since the limit is 5 and only 4 steps were used.

The run shows two things. The safety rule worked: a blocked call is returned as ordinary data, and the model recovered from it. It also shows a weakness. The model said "24 trials" but the log shows 25 rows were returned, so a model's summary should not be trusted without checking the tool result. The harness also wasted one call on a duplicate. A future improvement would be to detect repeated identical calls and stop them.

Self-check: Verification.json:

```

Rutuja Saste> python code\verify_hw5.py | Tee-Object -FilePath reports\hw05\RUN_LOG.txt
PASS FastAPI backend responds on port 8425
PASS trials MCP server lists 3 tools and answers a call ['search_trials', 'get_trial', 'count_trials_by_sponsor']
PASS meals MCP server lists 4 tools and answers a call ['search_meals_by_name', 'meals_by_ingredient', 'random_meal', 'meal_details']
PASS offline tests all pass ['11/11 tests passed']
PASS 150 fault-injection records saved 150
PASS agent_runs.jsonl exists
all passed: True

```

AI_USE.md

1. What I used an AI assistant for, and what I did myself

I used AI to explain the assignment, understand and write the code for Redux slice, MCP servers, retry logic and fix errors. I did all the validations against MCP server. Updated the SQL tables by including the sponsor table and linking it with the trials table. Did overall changes for the flow and also ran the verification tests.

2. One AI-produced output that was wrong or unsuitable

The AI gave me MCP server code using FastMCP, but `pip install "mcp[cli]"` installed MCP 2.x, where that class was renamed, so `mcp dev` failed with `No module named 'mcp.server.fastmcp'`. Other AI problems I hit: a stray period in AI-written comments in `seed_hw4.py` caused a `SyntaxError` on my second laptop; `jinja2` was missing from the AI's install list; and in scenario 4 the agent said "24 trials" while the tool result had 25 rows.

3. How I detected the problem

The MCP error message named the rename directly, and I confirmed the installed version with `pip show mcp (2.3.0)`. The other issues showed up as errors when I ran the code on a clean machine, and I counted the rows in `agent_runs.jsonl` to catch the "24 vs 25" mistake.

4. What I changed and why it works now

I pinned `mcp<2 (1.30.0)`, which has FastMCP, and launched the Inspector with my `venv Python` instead of `uv`; both servers now connect. I removed the stray periods, installed `jinja2`, and set `DATABASE_URL`, so the backend and seed run on both laptops. For the agent, I documented that a model's summary must be checked against the tool result.